Drupal 9 (or later) migrate modules

(Last updated on 20 February 2024)

This documentation needs work. See "Help improve this page" in the sidebar.
This page contains a list of the Drupal 9 (or later) core and most significant contributed modules.

Core Migrate modules

Please refer to Migrate API overview for technical documentation of core features.

Migrate

Provides the underlying API for migrating configuration and content to Drupal 9 (or later). The migration source may be a different system altogether.

Migrate Drupal

Provides the capabilities needed to migrate configuration and content from a Drupal source site to Drupal 9 (or later).

Migrate Drupal UI

Provides a user interface (at /upgrade) for performing an upgrade from Drupal 6 or Drupal 7 to Drupal 9 (or later). See Upgrading using web browser.

Contributed Migrate modules

Location Migration

- Migrates data from the Location module on Drupal 7 to the Address and Geolocation modules.

Migrate: Exclude Entities

- Provides a simple mechanism, using settings.php changes, to block specific entity types from being migrated during a core upgrade using either Migrate Drupal or Migrate Upgrade.

Migrate File Entities to Media Entities

- Allows migration of Drupal 7 Files entities to Media Entities

Migrate Media Handler

- Provides migration process plugins to facilitate conversion of Drupal 7 file/image fields and inline file embeds in rich text, into full-fledged Drupal 8+ media entities.

Migrate HTML to Paragraphs

- Migrate API module to turn HTML from the migration source into Paragraph items on the migration destination.

Migrate Manifest

- Provides a Drush command for running template-based migrations using a manifest format.

Migrate Plus

- Provides APIs for grouping migrations as well as a facility to manipulate incoming source data in migrations as well as code examples to build custom migrations. Required by Migrate Upgrade.

Migrate Process Markdown to HTML

- A process plugin to convert Markdown to HTML.

Migrate Scanner

- Provides a recursive directory scanner for migrations files, which helps to organize migrations into sub-directories.

Migrate: Skip on 404

- Allows core migrations, using either Migrate Drupal or Migrate Upgrade, to stop the normal file migration from failing if it comes across a file that cannot be located.

Migrate Source UI

- User interface for importing data from JSON, CSV or XML sources. Work with migrate tools and other migrate modules to allow someone to upload the source file.

Migrate Tools

- Since Drush 10.4, most Migrate Tools commands such as migrate:status, migrate:import, and migrate:stop are are included in Drush, except for the migrate:tree command which shows a tree of migration dependencies, which you can get by installing the module. The code-base that made its way into Drush was from an earlier and simpler fork of the Migrate Tools code. For example, Drush does not support migrate plus config entities or provide a progress bar. Over time, the Drush team continues to add features and some of this might become incorporated upstream.
- See Upgrade using Drush.

Migrate Upgrade

- Provides the migrate:upgrade a Drush command for generating the migrations from a Drupal source site.
- See Upgrade using Drush.

Migrate: URL2Link

- Provides a migration path for the URL module on Drupal 7 to the Link module in Drupal 8 & 9.

Migrate: Views

- Provides a migration path for the Views (for Drupal 7) module on Drupal 7 to the Core module - Views in Drupal 9.

Migration Tools

- Adds classes and methods that help simplify a number of migration tasks including parsing source HTML, processing strings, extracting selected source content, debugging and troubleshooting, and more.
- See: DrupalCon Seattle 2019 - Migrating terrible static content into Drupal 8

WordPress Migrate

- Can work with WordPress blog exports (WXR formats) and Drush to mass migrate from WordPress to Drupal 7 or Drupal 9 or later.
- Original documentation. Drupal 7 guide. Docs planned to move here eventually.

Tools & Modules For Upgrading

To assist you in the upgrade process, consider using the following tools and modules:

- Core Migrate Module Suite: For migrating your content and site configuration.
- Upgrade Status Module: Provides information about the availability of contributed projects.
- Upgrade Rector: Automates module updates for Drupal 8 sites with common deprecated code to the latest Drupal 9-compatible code.
- Drupal Module Upgrader: Scans and converts outdated code on your Drupal 7 site.

Migrate Tools (https://www.drupal.org/project/migrate_tools)

The Migrate Tools module provides tools for running and managing Drupal migrations.
Drush commands supported include:

- migrate:status - Lists migrations and their status.
- migrate:import - Performs import operations.
- migrate:rollback - Performs rollback operations.
- migrate:stop - Cleanly stops a running operation.
- migrate:reset-status - Sets a migration status to Idle if it's gotten stuck.
- migrate:messages - Lists any messages associated with a migration import.
- migrate:fields-source - List the fields available for mapping in a source
- migrate:tree - Shows a tree of migration dependencies

A **Web UI** provides a front-end equivalent to many of the drush commands.

Versions

Note:
Since Drush 10.4 many of the drush commands are also available in Drush core. The code-base that made its way into Drush 10.4 was from an earlier and simpler fork of this module's code. For example, Drush does not support migrate plus config entities or provide a progress bar. Over time, the Drush team continues to add features and some of this might become incorporated upstream. The support team for Drush is unique from migrate tools. Any features or support requests for Drush core should be handled in its Github issue queue.

Upgrade using Drush

Last updated on 24 October 2023
Drush is a command line shell and scripting interface for Drupal. Upgrading to Drupal 9 or higher using Drush is an alternative to using the browser user interface. Upgrading using Drush is very useful when migrating complex sites as it allows you to run migrations one by one and it allows rollbacks.

Installing Drush with Composer

Drupal 8 or higher sites can be built using Composer. The recommended Composer project doesn't have Drush listed as a dependency, you can install Drush from the command line as follows:
composer require drush/drush
This will give you the latest (stable) Drush which should be compatible with a recent Drupal. For more information about this, read the Drush and Drupal version compatibility matrix.
To check your version of Drush, use
drush --version

Required Migration modules

To run the upgrade with Drush you will need these two contrib modules:

- Migrate Upgrade: Use this module to use migrations as a configuration. This is a common practice but is not required by Drupal core.
- Migrate Plus: Provides extensions to core migration framework functionality, required by Migrate Upgrade.

You can download the two modules as follows:

composer require drupal/migrate_upgrade drupal/migrate_plus

Enable the modules:
drush install migrate migrate_drupal migrate_upgrade migrate_plus
Note: Since Drush 10.4, most Migrate Tools commands are included in Drush, except for the migrate:tree command which shows a tree of migration dependencies, which you can get by installing the module. The code-base that made its way into Drush was from

an earlier and simpler fork of the Migrate Tools code. For example, Drush does not support migrate plus config entities or provide a progress bar. Over time, the Drush team continues to add features and some of this might become incorporated upstream.
IMPORTANT: Pay close attention to selecting the correct version for each of the three contributed modules. Please refer to the project page for selecting a version compatible with your version of Drupal core.
For more information on the different upgrade modules, please refer to the list of upgrade modules.

Optional migration modules

Visit Drupal 9 (or later) migrate modules for a list of further migration modules.

Define the source database

This is an example of how to define the database connection details for your Drupal 6 / 7 source site. If your source site uses a database prefix, be sure to provide the prefix. You will need to connect to both the local development database (default) as well as the source database (D6 or D7). The following is an example of how to do this with Lando. The example includes the default database set up for completeness.
Note: It is important to name your migrate database key migrate to avoid issues, see Getting 'Field discovery failed for Drupal core version 7.' when migrating.

DDEV: Set up database and source credentials in settings.php
Start up both the source web site (Drupal 7) and target web site (Drupal 10) in separate DDEV instances. Enter the D7 credentials in the Drupal 10 settings.php, using the Docker containers listed during start up, for example ddev-drupal7-db and ddev-drupal7-web, shown during start up. See Upgrade using web browser > Example values for DDEV for an example.

Lando: Set up database and source credentials in settings.php

```
$databases['default']['default'] = [
  'database' => 'drupal8',
  'username' => 'drupal8',
  'password' => 'drupal8',
  'prefix' => '',
  'host' => 'database',
  'port' => '3306',
```

```
  'namespace' => 'Drupal\\Core\\Database\\Driver\\mysql',
  'driver' => 'mysql',
];

$databases['migrate']['default'] = [
  'database' => 'drupal7db',
  'username' => 'drupal7db',
  'password' => 'drupal7db',
  'prefix' => '',
  'host' => 'd7db',
  'port' => '3306',
  'namespace' => 'Drupal\\Core\\Database\\Driver\\mysql',
  'driver' => 'mysql',
];
```

Example .lando.yml

```
name: mywebsite
recipe: drupal8
config:
  webroot: web

# Create Drupal 7 database service, consider adding phpmyadmin
services:
  d7db:
    type: mariadb
    creds:
      user: drupal7db
      password: drupal7db
      database: drupal7db
    portforward: true
```

Import the Drupal 7 database, assuming the database dump file is called mywebsite_db.sql.gz and in the current folder:

lando db-import --host=d7db --user=drupal7db mywebsite_db.sql.gz

Inspiration from https://github.com/thinktandem/migration_boilerplate.

Generating migrations using migrate:upgrade

Drush adds commands such as drush migrate:status and drush migrate:import. A full list of the migrate-related Drush commands can be found at the bottom of this page.
If you try drush migrate:status without doing anything else you won't see any migrations available to run. That's because the individual migrations have to first be created based on your source database. Since Migrate has no idea what source to use, no migrations have been created yet.

To generate the migrations you will need the Drush command drush migrate:upgradewhich is provided by the contributed Migrate Upgrade module.
You most probably want to only generate individual migrations so that you can run them one by one. If this is the case, you need to use the --configure-only option.
The --legacy-db-key option allows you to use a $databases array, with a matching key, defined in settings.php:

drush migrate:upgrade --legacy-db-key=migrate --legacy-root=https://example.com --configure-only

'https://example.com' is the root of your source site. If the legacy site is on the local filesystem, you can use a file path to the Drupal root directory as the value for this option. The value that you supply here will be prepended to the paths of the individual files to locate and import them.
Use --legacy-db-url for a remote database:

drush migrate:upgrade --legacy-db-url=mysql://user:password@server/db --legacy-root=https://example.com --configure-only

where

- 'user' is the username of the source database
- 'password' is the source database user's password
- 'server' is the source database server
- 'db' is the source database

If your source site uses a table prefix in the database table names, you will need to add that as an additional argument as follows. In this example, the database prefix is 'drupal_'

drush migrate:upgrade --legacy-db-url=mysql://user:password@server/db --legacy-db-prefix=drupal_ --legacy-root=https://example.com --configure-only

If you don't use the --configure-only option, drush migrate:upgradewill first generate and then execute all migrations.
After running migrate:upgrade with the --configure-only parameter, you run migrate:status to see the list of possible migrations:
drush migrate:status

Then you can review and selectively execute these migrations. To perform the migrations individually run the:
drush migrate:import <migration name>

There are several Drupal 6 and Drupal 7 migrations that are always created. If you are upgrading from a Drupal 6 site you will see some Drupal 7 migrations when running some drush commands, such as drush migrate:status, and vice versa if you are upgrading from Drupal 7 you will see some Drupal 6 migrations when running some drush command. To avoid seeing those simply use the --tag= option on the drush command:
drush migrate:status --tag='Drupal 7'

To perform all the migrations in the list, run the:
drush migrate:import --all

To perform only Drupal 7 migrations, run:
drush migrate:import --tag='Drupal 7'

Alternatively, if you followed the guide Customize migrations when upgrading to Drupal 9 (or later), and created a custom migration module, you can also perform the migration with an additional group annotation:
drush migrate:import --group=your_module --continue-on-failure

This way you only migrate the migrations you defined in your custom migration modules config/install folder.
The --continue-on-failure will additionally prevent your migration from canceling after a failed migration. This way you can do a full migration, check the failed migrations, and implement custom processing for the failed migrations inside your custom module.

Running specific migrations using migrate-manifest

It is also possible to use a manifest file to set up a specific set of migrations. This lets you run groups of migrations in a reproducible manner. For this method, you also need the Migrate Manifest module. With Migrate Manifest, you can get a complete list of available migrations using the following commands:
drush migrate:template:list

The desired migrations are defined as a YAML file as shown in the example below. You only need to list the migrations you need. Migrate Manifest will ask you to add additional migrations that are needed to resolve any dependencies. Migrations can be listed in any order, they will be executed in the correct order based on the dependencies.

```
# user
- d6_user
- d6_user_profile_field
- d6_user_profile_field_instance
- d6_user_profile_entity_display
- d6_user_profile_entity_form_display
- d6_profile_values:user
- d6_filter_format
- d6_user_role
- d6_user_picture_entity_display
- d6_user_picture_entity_form_display
- d6_user_picture_file
- d6_user_picture_field
- d6_user_picture_field_instance

# taxonomy
- d6_taxonomy_vocabulary
- d6_taxonomy_settings
- d6_taxonomy_term

# nodes
- d6_node
- d6_node_revision
- d6_node_type
- d6_view_modes
- d6_filter_format
- d6_field_instance_per_form_display
```

```
- d6_field_instance_widget_settings
- d6_field_formatter_settings
- d6_field_instance
- d6_field
- d6_field_settings
- d6_node_settings
- d6_cck_field_values:*
- d6_cck_field_revision:*

# taxonomy fields
- d6_term_node_revision
- d6_term_node
- d6_vocabulary_entity_display
- d6_vocabulary_entity_form_display
- d6_vocabulary_field_instance
- d6_vocabulary_field

# blocks
- d6_block
- d6_menu

# custom blocks
- d6_custom_block
- d6_filter_format

# book
- d6_book
- d6_book_settings

# file migrations are configurable, see https://www.drupal.org/node/2257723
- d6_file:
    source:
      conf_path: sites/assets
    destination:
      source_base_path: destination/base/path
      destination_path_property: uri
```

Place the manifest file in a location that is accessible when running Drush. Storing it in your version control system is recommended so that you can track changes to your migrations.
Make sure that modules used by the migrations listed in the manifest file exist and are enabled on your source site (e.g. field module for d6_field) Otherwise, there will be errors when the migrations are executed.

The migrations defined in the manifest file are executed from the command line as shown below. Replace the database URL and path to manifest file with appropriate values (like migrate:upgrade, migrate-manifest will accept a MySQL URL or settings.php array key):
drush migrate-manifest
--legacy-db-url=mysql://d6user:d6pass@localhost/drupal_6 manifest.yml

Notes for Acquia Dev Desktop users

If you use Acquia Dev Desktop and have your Drupal 6 site in Dev Desktop, the default database credentials are drupaluser with an empty password and the 33067 port for the database on the 127.0.0.1 IP address. Altogether that leads to --legacy-db-url=mysql://drupaluser:@127.0.0.1:33067/drupal_6 the commands assuming the database name is drupal_6. Run drush status if you are having trouble connecting in order to verify these values.

Reference of Drush migration commands

migrate:upgrade (no alias)

Provided by the Migrate Upgrade project. Use this to run an upgrade from Drupal 6 / 7. This command will generate migration configurations based on the source site's configuration and content.
Refer to the examples earlier on this documentation page.

Basic example

drush migrate:upgrade --legacy-db-key=migrate

Options

- legacy-db-url: Database connection information for the source database.
- legacy-db-prefix: Database table prefix for the source database.

- legacy-root: Path to the source site, this is used to transfer content from the files directory. If the files are private then a local file path must be specified, for public files http(s) will work as well.
- configure-only: Use this to create migration configurations only. When this option is set, migrations will be generated so that they can be executed individually with 'drush migrate:import'.

migrate:status (ms)

Included in Drush since 10.4.0. Use this to get the list of all migrations with their current status. Use the --group or --tag option to filter the results.

Basic example

drush migrate:status

migrate:import (mi)

Included in Drush since 10.4.0. Use this to perform one or more migration processes. This is normally used with custom migrations from non-Drupal sources. For example, if you've created and imported a custom migration configuration, this command can be used to run it.

Basic examples

drush migrate:import migration_id
drush migrate:import --tag='Drupal 7'
drush migrate:import --group=files

Options

- all: Run all migrations currently configured.
- group: Run all migrations belonging to a particular group.
- limit: Limit the number of items to process in each migration.
- feedback: Frequency of progress messages, in items processed.
- idlist: Comma-separated list of source IDs to import.
- update: Migrate new items as well as previously migrated items that have been updated on the source.
- force: Force an operation to run, even if all dependencies are not satisfied.

Common use cases

- When migrate:upgrade --configure-only is used, configuration entities are created. The default behavior of the new Drupal site is to not allow existing configuration entities to be overwritten, migrate:upgrade cannot be used to run a migration after migrate:upgrade --configure-only is used. Use migrate:import instead.
- When a custom migration is created and imported (either via the configuration management UI or drush config:import), use migrate:import to run the migration.
- Use Migrate Devel for troubleshooting errors during import. Example: drush migrate:import migration_id --migrate-debug-pre.

Pro tip: If you've imported a custom migration configuration and you need to update it and re-import it, use the Configuration Update Manager module.

migrate:rollback (mr)

Included in Drush since 10.4.0. Use this to roll back a migration. This is normally used for testing or if you had issues and need to start over. Use this with the id or group of the migration task.

Basic examples

drush migrate:rollback migration_id
drush migrate:rollback --tag='Drupal 7'
drush migrate:rollback --group=files

migrate:stop (mst)

Included in Drush since 10.4.0. Use this to stop an active migration operation.

Basic example

drush migrate:stop migration_id

migrate:reset-status (mrs)

Included in Drush since 10.4.0. Use this to reset an active migration status back to idle.

Basic example

drush migrate:reset-status migration_id

migrate-manifest (no alias)

Provided by the Migrate Manifest module.

migrate:messages (mmsg)

Included in Drush since 10.4.0. Use this to view any messages associated with a migration. This is useful to use when you have a failure in your migration process. This will show you why they failed and what caused them.

Basic example

drush migrate:messages migration_id

migrate:fields-source (mfs)

Included in Drush since 10.4.0. Use this to list the fields available for mapping in a source.

Basic example

drush migrate:fields-source migration_id

migrate:tree (no alias)

Provided by Migrate Tools module, the only command not yet included in Drush. Use this to show a tree of migration dependencies.

Basic example

drush migrate:tree --tag='Drupal 7'

Upgrade using web browser

Last updated on 26 October 2023
Where the images refer to Drupal 8, it also applies to Drupal 8 or higher.

Before you begin

Before you begin the upgrade, make sure you have read the previous pages of this handbook:

- Upgrading from Drupal 6 or 7 to Drupal 9 (or later)
- Preparing an upgrade
- Choosing the upgrade approach

Make sure you are using the latest version of Drupal 9 or higher. You can check your version at the status report page of your Drupal site (/admin/reports/status).
Make sure the following core modules are enabled on your new Drupal site:

- Migrate
- Migrate Drupal
- Migrate Drupal UI

Install and enable the contributed modules on the new Drupal site for which you want to migrate configuration and content from the source site.

Start

You can start the upgrade process by visiting the /upgrade path of your new Drupal site If you get a Page not found error, verify you have enabled the 'Migrate Drupal UI' module.
If an upgrade was not done then the Upgrade page provides information about the preparation steps that should be done before continuing with the upgrade. However, if an upgrade was done then you are asked if you want to import new configuration and content. If you continue, for either of these situations, you are then asked to define the source site.

Define the source site

- Define the database connection details for your Drupal 6 / 7 source site. If your source site uses a database prefix, be sure to expand the Advanced options and provide the prefix.
- Define public and private files directories of your source site so that files can be copied to your new Drupal site.
- You can define these values in settings.php, see below.
- See screenshot below.

Drupal version of the source site *

- ○ Drupal 6
- ◉ Drupal 7

▼ **SOURCE DATABASE**

Provide credentials for the database of the Drupal site you want to upgrade.

Database type *

- ◉ MySQL, MariaDB, Percona Server, or equivalent
- ○ PostgreSQL
- ○ SQLite

Database host *

Database name *

Database username *

Database password

▶ **ADVANCED OPTIONS**

▼ **SOURCE FILES**

Public files directory

To import public files from your current Drupal site, enter a local file directory containing your site (e.g. /var/www/docroot), or your site address (for example http://example.com).

Private file directory

To import private files from your current Drupal site, enter a local file directory containing your site (e.g. /var/www/docroot).

Review upgrade

Review the pre-upgrade analysis

If the database credentials to your source database are correct, the upgrade review page will appear as illustrated in the screenshot below. This page shows a summary of the upgrade status for all installed modules on the old site.

What will be upgraded?

Home

5 MODULES WILL NOT BE UPGRADED
Details

33 MODULES WILL BE UPGRADED
Details

MODULES THAT WILL NOT BE UPGRADED

There are no modules installed on your new site to replace these modules. If you proceed with the upgrade now, configuration and/or content needed by these modules will not be available on your new site. For more information, see Review the pre-upgrade analysis in the Upgrading to Drupal 8 handbook.

DRUPAL 7	DRUPAL 8
addressfield	Not upgraded
color	Not upgraded
locale	Not upgraded
rdf	Not upgraded
views	Not upgraded

MODULES THAT WILL BE UPGRADED

DRUPAL 7	DRUPAL 8
block	block block_content
comment	comment
contextual	core

As a site builder you should carefully review the modules that will not be upgraded. For each module, evaluate if your new Drupal site will work without the module. The list

includes modules that are installed on the old site but not on the new site and modules that do not provide an upgrade path.
Let's use some of the modules shown in the screenshot above as examples.

- Addressfield provides a field type for storing address data.
- If you would proceed with the upgrade, the nodes that had addresses on your source site would be migrated to the new Drupal without addresses which would most probably not be the desired outcome.
- This issue can be resolved by installing and enabling the Drupal Address module.
- In other words, you would not proceed and perform the actual upgrade but you would first install the missing Drupal module and then start the upgrade process again. You would then see that the Addressfield is included in the list of modules that will be upgraded.
- Color is a Drupal 7 core module that allows to easily change the color scheme of compatible themes.
- When the screenshot was made there was no upgrade path for migrating the configuration settings of the Color module from Drupal 7 to Drupal 9 or higher..
- However, you can manually set the configuration settings on your new Drupal site after the upgrade.
- It was safe to proceed with the upgrade and make a note that the configuration settings of the Drupal 7 module will not be automatically migrated.
- Views allows you to create different kinds of lists of your site content.
- Even if you have Views enabled on the new Drupal site it will appear as 'Not upgraded'. This is because there is no automatic upgrade mechanism that would migrate your Views configurations from Drupal 6 / 7 to Drupal 9 or higher.
- It is safe to proceed with the upgrade but you will need to manually create the views on your new Drupal site after you have executed the upgrade.

The list of modules that will be upgraded is collapsed by default. When you open the list, you can review what will end up where on the new site. For example, this shows that the block module from the source site is migrated to two modules on the new Drupal site. There is absolutely nothing to do about these, the list is purely informational.

▼ MODULES THAT WILL BE UPGRADED

This is a list of the modules that will be upgraded and the Drupal 8 module or modules it will be upgraded to.

DRUPAL 7	DRUPAL 8
✔ block	block block_content
✔ date	datetime
✔ dblog	dblog

ID conflict warnings

As described in the Preparing an upgrade page, upgrading from Drupal 6 or Drupal 7 to Drupal 9 or higher should be done to an complete empty new Drupal site. For example, if you manually create a node to the new Drupal site before upgrading and the source Drupal 6 / 7 site has a node with the same ID, the migration system will overwrite the node that was manually created in new Drupal site.
If conflicting IDs are detected, a warning about conflicting IDs will be shown as illustrated below. You can either ignore this warning and risk losing data or abort and take an alternative approach as described in the Known Issues documentation.

Perform the upgrade

The Pre-upgrade analysis summary is the final page before the actual execution of the upgrade, so this is the final chance to hit Cancel. Once you press the Perform upgrade button, the process is started.
Depending on the size and types of content/configuration on the source site, the upgrade may take a very long time (possibly hours for a bigger data set). Due to password encryption, user accounts take particularly long to import. Continuous feedback is provided along the way with both progress information and errors encountered:

Currently upgrading Nodes (Feed) (29 of 40 total tasks)
Upgraded Nodes (Book page) (processed 11 items total)
Upgraded User accounts (processed 1 item total)
Migration user_picture_entity_form_display did not meet the requirements. Missing migrations user_picture_field_instance. requirements: user_picture_field_instance.
Operation on User picture form display configuration failed
Migration user_picture_entity_display did not meet the requirements. Missing migrations user_picture_field_instance. requirements: user_picture_field_instance.
Operation on User picture display configuration failed
Migration user_picture_field_instance did not meet the requirements. Missing migrations user_picture_field. requirements: user_picture_field.
Operation on User picture field instance configuration failed
Source ID 986: File 'files/LocalizationClientDrupal8Alpha1Submission.png' does not exist.
Source ID 981: File 'files/LocalizationClientDrupal8Alpha1.png' does not exist.
Source ID 976: File 'files/Drupal8MenuInSidebar.png' does not exist.
Source ID 971: File 'files/Drupal8MenuItems.png' does not exist.
Source ID 966: File 'files/Drupal8FrenchArticle.png' does not exist.
Source ID 961: File 'files/Drupal8EditArticle.png' does not exist.
Source ID 956: File 'files/Drupal8ArticleFieldsCustomized.png' does not exist.
Source ID 951: File 'files/Drupal8MultilingualModulesOverlap.png' does not exist.
Source ID 946: File 'files/Drupal8TranslationSearch.png' does not exist.
Source ID 941: File 'files/Drupal8OutdatedWorkflow2.png' does not exist.
Source ID 936: File 'files/Drupal8OutdatedWorkflow1.png' does not exist.
Source ID 931: File 'files/Drupal8TranslationPermissions.png' does not exist.
...

70%

New messages show up at the top of the page, while old ones disappear at the bottom as the process goes on.

Review the migration logs

All of these messages shown during the upgrade can be reviewed later. Once the process is finished, you are directed to the site's frontpage with messages summarizing the results:

Completed 37 upgrade tasks successfully.

3 upgrades failed

Upgrade process not completed

Review the detailed upgrade log

Here you have a chance to review the messages that were logged and inspect any errors found. The module uses the built-in logging system of Drupal, so in a default setup you can just review the logs stored in the database, which is where the link leads to:

Recent log messages

▼ Filter log messages

Type

cron
migrate_upgrade
php
system
user

Severity

Emergency
Alert
Critical
Error
Warning
Notice
Info
Debug

Filter Reset

▶ Clear log messages

	Type	Date ▼	Message	User	Operations
	migrate_upgrade	02/02/2016 - 15:14	Upgraded Node title label (processed 6 items total)	Gábor Hojtsy	
	migrate_upgrade	02/02/2016 - 15:14	Upgraded Node revisions (Tweet) (processed 0 items...	Gábor Hojtsy	
	migrate_upgrade	02/02/2016 - 15:14	Upgraded Node revisions (Post) (processed 834 items...	Gábor Hojtsy	
	migrate_upgrade	02/02/2016 - 15:14	Upgraded Node revisions (Issue) (processed 0 items...	Gábor Hojtsy	
	migrate_upgrade	02/02/2016 - 15:14	Upgraded Node revisions (Feed item) (processed 0 items...	Gábor Hojtsy	

You can filter the list of messages to show notices (used for positive feedback) or errors (used for reporting problems found in the migration). If something breaks, you've either found a bug (in which case, check the issue queue or contact the Migration team) or your data set is too large (in which case, try running the upgrade with Drush).

Set default values in settings.php for the Migrate Drupal UI, /upgrade

When using the UI you must enter the database credentials for the source database and, if you are migrating files, the path to the public and private files.
These values can be set in settings.php so they do not need to be re-entered every time you perform an upgrade.

```
$settings['migrate_source_connection'] = '';
$settings['migrate_source_version'] = '';
```

```
$settings['migrate_file_public_path'] = '';
$settings['migrate_file_private_path'] = '';
```

where

- migrate_source_version - The version of the source database. This can be '6' or '7'. Defaults to '7'.
- migrate_source_connection - The key in the $databases array for the source site database.
- migrate_file_public_path - The location of the source Drupal 6 or Drupal 7 public files. This can be a local file directory containing the source Drupal 6 or Drupal 7 site (e.g /var/www/docroot), or the site address (e.g http://example.com).
- migrate_file_private_path - The location of the source Drupal 7 private files. This can be a local file directory containing the source Drupal 7 site (e.g /var/www/docroot), or empty to use the same value as Public files directory.

See change log record Set default values in settings.php for the Migrate Drupal UI, /upgrade.

Example values for DDEV

Assuming that your Drupal 7 site DDEV URL is drupal7.ddev.site the values could be something like this:

```
$settings['migrate_source_connection'] = 'migrate';
$settings['migrate_source_version'] = '7';
$settings['migrate_file_public_path'] = 'http://ddev-drupal7-web';
$settings['migrate_file_private_path'] = '';

$databases['migrate']['default'] = [
 'database' => 'db',
 'username' => 'db',
 'password' => 'db',
 'host' => 'ddev-drupal7-db',
 'port' => '3306',
 'driver' => 'mysql',
 'prefix' => '',
];
```

Preparation

Last updated on 24 September 2022

This documentation **needs work**. See "Help improve this page" in the sidebar.

Prior to upgrading it is recommended that you take a few steps to ensure the process will be smooth and have the desired results.

When upgrading from Drupal 6 or Drupal 7 *configuration* and *content* entities are *migrated* from the source site to a clean Drupal site. Refer to the Upgrading from Drupal 6 or 7 page for an overview of the process and key differences.

Even though the upgrade does not modify the source site in any way, it's recommended that it is not performed on a live site. It's a good idea to use a backup of your live site to ensure a consistent database state (no writes to the database while the migration is running), and to prevent any performance impact on the live site.

Module inventory of the source site

It is important to understand how your existing Drupal 6 or Drupal 7 site is built before you start your upgrade.

Identify the core and contributed modules that are enabled on your source Drupal 6 / 7 site. To do this, you can either look at the listed modules on the Modules admin page (Administer > Site building > Modules) or use the Available Updates page at admin/reports/updates.

Write down the complete list of modules enabled on your source site and use this as a checklist to plan your upgrade. For each module, answer the following questions:

- Do I still need this module on Drupal 9?
- Has the contributed module moved to core? For example Views is in core since Drupal 8.
- Does the contributed module have a Drupal 9 version available? If not, do I still need it? Are there other modules that I could use to achieve the same functionality?

- Please note that the Drupal 6/7 modules do not necessarily map one to one to Drupal 9 or higher modules. For example, the Block module in Drupal 6 and 7 was separated into the Block and Custom Block modules since Drupal 8.

The Drupal upgrade web user interface shows a summary of which modules can be automatically upgraded and which ones cannot. Refer to Upgrade using web browser page for more information on this analysis.

You can use the Upgrade Status module to check the list of projects you have installed, and show their availability.

Check the Known Issues page

It is recommended to read through the Known Issues page when preparing the upgrade.

Update your Drupal 6 / 7 site to use the latest versions of core and contributed modules

The upgrade process expects you to first update your old site to the latest minor version. If your site is for example running Drupal core 6.35, you need to first update to 6.38.

If you're planning to automatically migrate configuration or content from contributed modules, you are expected to be using the latest versions of them.

Use the Available Updates page of your Drupal 6 / 7 site at at admin/reports/updates to see if there are updates available.

Pre-requisites for upgrading from Drupal 6 / 7

- A fresh installation of Drupal with the core modules 'Migrate' and 'Migrate Drupal' enabled. If you're upgrading using the browser user interface, you will also need to enable 'Migrate Drupal UI' core module.
- Access to the Drupal 6 or 7 database from the host where your new Drupal site is.
- Access to the source site's files.
- If only public files are used, they will be accessible through the site's address.
- If private files need to be migrated, the files directory needs to be directly accessible to the new Drupal site and you must configure the Drupal file_private_path path in settings.php before running the upgrade.
- If you're upgrading to Drupal 9 using Drush, you need to download and enable the following contributed modules.
- Installation instructions for Drush are included on the page linked above.
- Migrate Upgrade: Provides Drush support for upgrading from Drupal 6 or 7.
- Migrate Plus: Provides extensions to core migration framework functionality.
- For more information on the modules listed above, please refer to the list of upgrade modules.
- **IMPORTANT**: Pay close attention to the compatibility of the module versions and the version of Drupal core you are using. The module pages of the three modules listed above clearly state which version to use.

Enable required core and contributed modules

The migration process does **not** automatically install modules on the Drupal destination site. Only migrations relevant for modules installed on both the source and destination site will be executed. Therefore you need to first enable all core and contributed modules on the Drupal site for which you want to migrate configuration and content from the source site.

A couple of examples:

- If you have the Addressfield module enabled on your source Drupal 7 site and want to migrate the address data, you must enable the Address module on your Drupal destination site before beginning the upgrade process. This is a good example where the modules do not map 1:1 between Drupal 7 and later versions of Drupal. However, the Address module provides the migrate plugins that are needed to migrate the data from Drupal 7 Addressfield.
- If you have the Drupal 7 core Forum module enabled on your source site, you must enable the core Forum module on your Drupal destination site before beginning the upgrade process.

Do not configure the Drupal destination site

Keep in mind that the upgrade process will overwrite configuration on the Drupal destination site, so **do not do any configuration of the Drupal site until after the upgrade process is complete**. This means you do not have to create all the content types and fields manually before running this upgrade. The Migrate module will create them as a part of the migration process. In fact, it is a common best practice to install using the Minimal install profile, because the Standard profile introduces its own configuration that you might not want to use. This is different than the way Migrate worked in Drupal 7 and earlier, where you had to have all the content types and fields prepared and ready before running Migrate.

Do not create content on the Drupal site

The upgrade process maintains the unique identifiers from the source site. For example, if a user account had the unique ID 2 on the source site (thus was at the relative URI /user/2), after upgrade the same user account will be at /user/2 on the new site. If you were to create a user account on the new site beyond the default administrator account, it would be assigned user ID 2 - then, when running the upgrade process, the account from the source site with user ID 2 would overwrite it. This applies to user accounts, taxonomy terms, and node content, as well as pretty much any other kind of content with numeric IDs from your source site.

To avoid data loss, do not manually create anything on the Drupal site until all your source content has been fully imported.

Performing the upgrade

You have two choices to actually execute the migration:

1. Using the browser user interface. This is the easier method and it can be done through the user interface. However, it offers less control over the migration.
2. Using Drush. This is more robust and faster and allows selective migration but it requires additional modules and it must be run from the command line.

Choosing the upgrade approach

When preparing an upgrade, it is worth spending a while to plan the upgrade approach. While the *content* of your previous Drupal site can most probably be migrated without issues, the *configuration* of your new Drupal site will most probably need some manual tasks. For example, Views module doesn't have an automatic upgrade path in core which means you will need to manually create the views on your new Drupal site. The most suitable approach needs to be considered for each site.

Simple approach with manual post upgrade steps

In this approach the upgrade is executed once and the manual post upgrade steps (e.g. creating the views) are performed on the new Drupal site before publishing it.

It is recommended to practice the upgrade first and take careful notes on the manual post upgrade steps. Once you are happy with your test environment, repeat the exact same steps and build your new Drupal production environment to a clean Drupal installation.

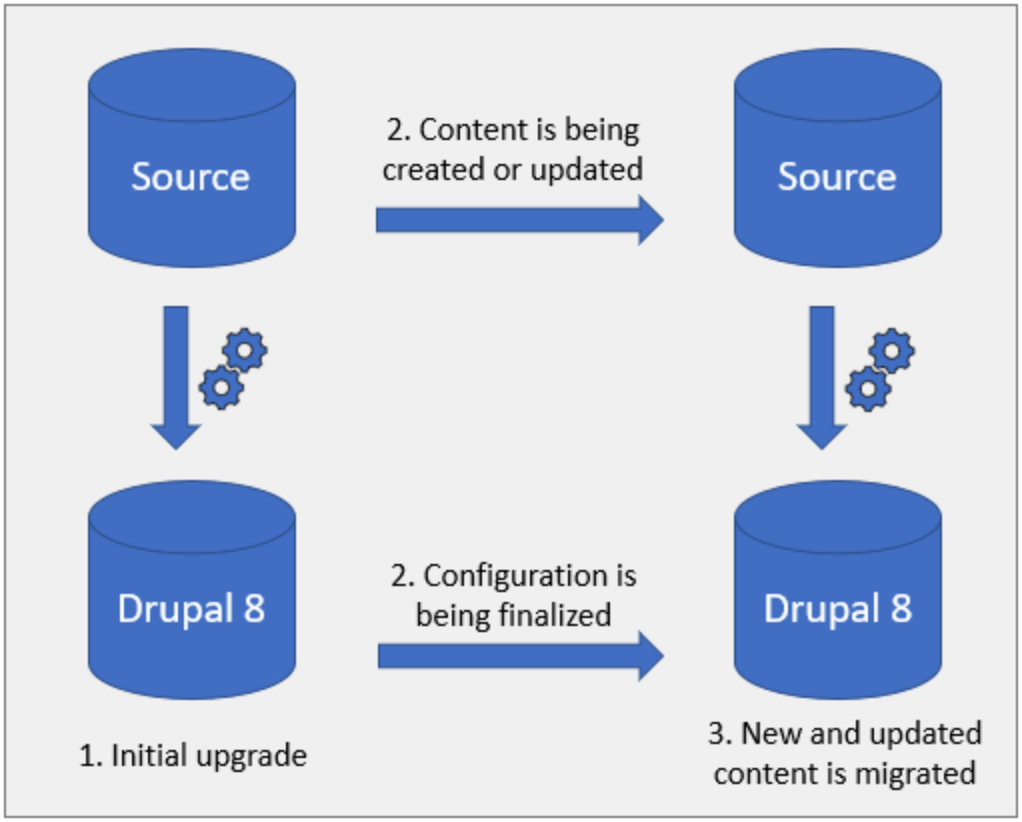
This approach can be applied by

- Upgrading using web browser
- Upgrading using Drush

Incremental upgrade after manual post upgrade steps

Depending on the amount and complexity of the manual post upgrade steps, it might not be feasible to keep the old site offline when the manual post upgrade steps are being executed on the new Drupal site. If the old site is online when the manual post upgrade steps are being done on the new Drupal site, it is possible that the users are creating or updating content on the old site.

Drupal supports *incremental upgrades*. This means that the migrations are executed again in order to migrate new and updated content. It is not recommended to change the configuration of the old source site between the initial upgrade and incremental migration.



This approach can be applied by

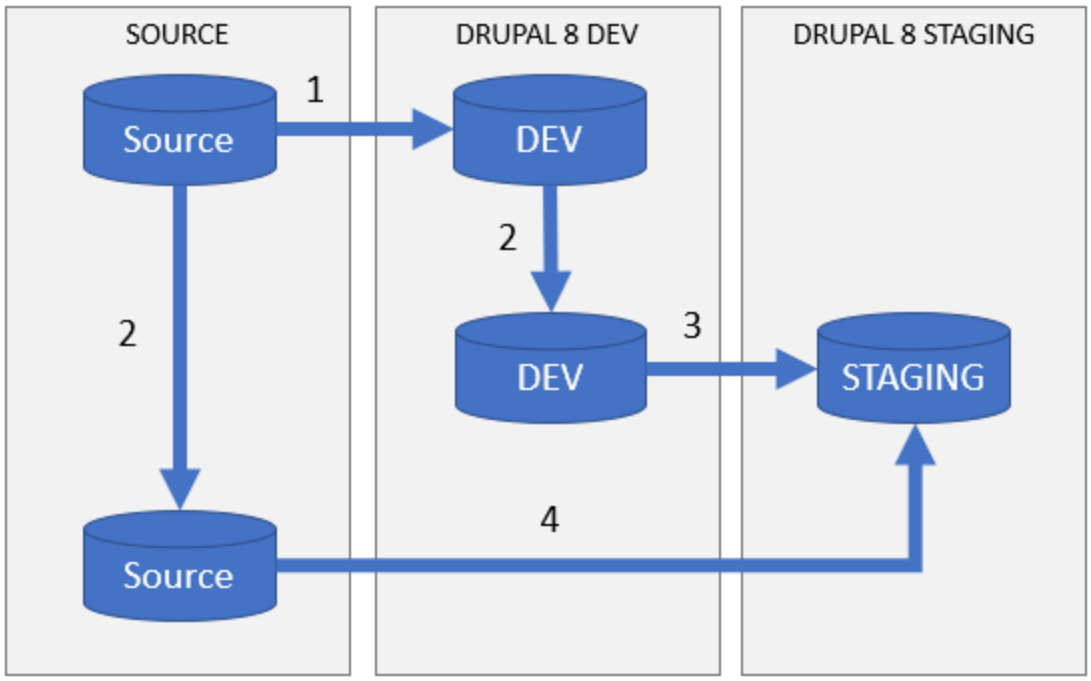
- Upgrading using web browser
- Upgrading using Drush

Migrating content only to STAGING and PROD environments

One of the most powerful new concepts in Drupal is an advanced configuration management system that can be used to deploy the whole Drupal configuration from one environment to another. Perhaps the most typical environment landscape is to have a development environment (DEV), a test environment (STAGING) and a production environment (PROD) which is the live site for end users. In Drupal 8 or higher, all configurations can be exported as YAML files and imported to another environment.

If you are planning to have separate DEV, STAGING and PROD environments, one possible approach is illustrated in the picture below. It is important to understand that the Drupal entities can be categorized as follows:

- Content entities (users, nodes, taxonomy terms, ...)
- Configuration entities (content types, fields, views, all configuration settings)



The upgrade process in the picture above:

1. The new Drupal DEV environment is upgraded from the source site.
2. The source site is kept online (content can be created or updated). At the same time, the configuration is finalized on the new Drupal DEV environment.
3. The configurations can be deployed from DEV to STAGING using the new Drupal configuration management system. Version control system (e.g. Git) is typically used.

4. When the configuration of the STAGING environment is complete, *content* can be migrated from the source site to STAGING for final acceptance testing. Once the acceptance tests are OK, the new Drupal PROD site is deployed and *content* is migrated from the old site to new Drupal PROD site.

This approach requires upgrading using Drush. There are a couple of possible variants to this approach.

- Refer to upgrading using Drush page on how to generate the migrations.
- The upgrade to the new Drupal DEV (step 1 in the picture above) can be done by executing all migrations with `drush migrate-import --all` or by executing the desired migrations one by one using `drush migrate-import <migration name>`
- Upgrading content only (step 4 in the picture above) can be done by executing all content migrations with `drush migrate:import --tag=Content` or by executing the desired migrations one by one using `drush migrate-import <migration name>`. It is also possible to list the desired migrations in a manifest file.

Customize migrations

Last updated on 28 November 2022

Create your initial migrations

- Create the migrations using `drush migrate:upgrade --configure-only` as discussed in Upgrade Using Drush
- Make sure you have a `config/sync` directory, to which the next step will write
- Export the migrations using `drush config:export`
- Create your custom migration module
- Copy only the YML files you want to use from the `config/sync` directory into the `config/install` directory of your new custom module, editing them to remove the `uuid` values and to edit the id, group, label, and other values as needed
- Copy the group file the `migrate_plus.migration_group.migrate_drupal_7.yml` from `config/sync` to the `config/install` directory of your new custom module, naming it `migrate_plus.migration_group.your_module.yml`. You will need the group file as it contains the database settings.

Migration management

- Before installing your custom migration module, make sure all the generated migration configs from `drush migrate:upgrade --configure-only` are deleted (Run `drush config:export`, delete the `migrate_plus.*` config files and run `drush config:import`).
- Now install your custom migration module and all your filtered YML files from your modules `config/install` folder gets installed.
- Now run `drush migrate:import --group=your_module --continue-on-failure` to run your migration.
- Note, that if you are using the Migrate Tools module, you can run `drush migrate:tree` to output the migration dependency tree. But generally, the `drush migrate:import --group=your_module --continue-on-failure` command will run the migrations in the correct order specified by the migration YML's "migration_dependencies" entry.
- As you proceed with adding more files to your module's config/install directory, re-read the directory with something like `drush config-import --partial --source=modules/custom/your_module/config/install`.

Rename fields when upgrading

Let's say that you have *content type A* in your Drupal 7 site with fields *foo*, *bar,* and *baz*. Let's also say that you want to rename the field *baz* to *zot*. These kinds of changes are quite simple to do when you are migrating using drush.

- Create the migrations using `drush migrate:upgrade --configure-only` as instructed on the page linked above.
- Run the node type and field migrations. This will generate *content type A* and fields *foo*, *bar,* and *baz*.
- Manually create field *zot* on the destination site. Delete the field *baz* that the migrations created but which you are not planning to use.
- The contributed Migrate Plus module allows migration plugins to be implemented as configuration entities, allowing them to flexibly be loaded, modified, and saved. The individual migrations generated with `drush migrate:upgrade --configure-only` can now be inspected by navigating to admin/config/development/configuration/single/export and by selecting 'Migration' as the Configuration type. Select the migration for *Nodes (A)*.
- Copy-paste the migration to admin/config/development/configuration/single/import but change the field mapping so that the destination field will be *field_zot* but you still map the source from *field_baz*. The exact content of the migration definition will depend on the field type. To understand the anatomy of the migrations, please refer to the examples provided in the Migrate API documentation.
- Once you have imported the modified migration, you can run the migration for A nodes with Drush and verify that the data was migrated to field *zot* correctly.
- Repeat the mapping change for Node revisions (A) migration if you're planning to also migrate node revisions.

Another way to achieve the same result is to manually create the new field *zot* as described above and alter the field mapping of *Nodes (A)* migration by implementing hook_migration_plugins_alter().

Writing custom process plugins

If you need to define custom transformation logic (if this, then that) you might want to consider writing a custom process plugin.

The example in Migrate API documentation uses User migration as an example. The user's language code determination needs if-else logic as described in the example and therefore the User module provides its own process plugin UserLangocode.

The example linked above describes how the process should be saved to the MODULE/src/Plugin/migrate/process directory and how it should be annotated so that it can be used. Use the UserLangcode and other process plugins as examples when writing your own!

hook_migrate_prepare_row()

If you need to define custom transformation logic, you can also implement hook_migrate_prepare_row() in your custom module and implement your logic in there.

As the API documentation of the hook describes, the hook has three arguments:

- Row $row
- MigrateSourceInterface $source
- MigrationInterface $migration

As described on the API documentation of the hook, you can use `$migration->id()` to limit your logic to only the desired migration.

To read a source property to a variable, use Row::getSourceProperty(), for example, if the source row would have a property called 'type', you could say `$type = $row->getSourceProperty('type');`

To set a completely new property that you can use in your migration definition as a source, use Row::setSourceProperty().

To make database queries to the source site, use `$source->getDatabase()->query()` as demonstrated in the hook API documentation.

To skip the row from being migrated, you can throw a new MigrateSkipRowException.

hook_migration_plugins_alter()

Another way to customize the migrations is to implement hook_migration_plugins_alter() in your custom module.

As described in the API documentation, this hook has one argument which is an associative array of all discovered migrations. You can alter the migrations or remove migrations that you don't want to be executed.

Migrate Plus provides a PREPARE_ROW event

If you prefer to write your custom responses to the core prepare_row hook in an object-oriented way instead of implementing the hook_migrate_prepare_row(), you can do this if you have the contributed Migrate Plus module enabled. Migrate Plus provides a PREPARE_ROW event which your event subscriber can register to. Drupal API documentation for Events. For an example of event subscriber implementation in another context, you can refer to this example from the Simple FB Connect module handbook.

Migrate and Migrate Plus Events

A (most complete) list of the migrate events you can react on :

```
MigrateEvents::MAP_SAVE
MigrateEvents::MAP_DELETE
MigrateEvents::POST_IMPORT
MigrateEvents::POST_ROLLBACK
MigrateEvents::PRE_ROW_SAVE
MigrateEvents::POST_ROW_SAVE
MigrateEvents::POST_ROW_DELETE

MigratePlusEvents::PREPARE_ROW
```

A useful implementation can be read in the constructor of `class MigrateExecutable,` Migrate Tool module.

Writing migrations for contributed and custom modules

Last updated on 29 December 2021

This documentation **needs work**. See "Help improve this page" in the sidebar.

This page complements Migrate API Overview by providing practical examples that can be used when writing migrations for contrib and custom modules.

Example: Migrating Configuration

A detailed example of how to migrate a Drupal 6 configuration variable to a Drupal 8 configuration object is provided in Migrating configuration entities.

Example: User Migration using Process Plugins and Process Pipelines

This example shows how the core User module is migrating users from Drupal 7 to Drupal 8. The migration is defined in `core/modules/user/migrations/d7_user.yml`. Let's break this into pieces as we did with the first example:

id: d7_user
label: User accounts

```
migration_tags:
  - Drupal 7
class: Drupal\user\Plugin\migrate\User
source:
  plugin: d7_user
process:
  # If you are using this file to build a custom migration,
  # consider removing the uid field to allow
  # incremental migrations.
  uid: uid
  name: name
  pass: pass
  mail: mail
  created: created
  access: access
  login: login
  status: status
  timezone: timezone
  langcode:
    plugin: user_langcode
    source: language
    fallback_to_site_default: false
  preferred_langcode:
    plugin: user_langcode
    source: language
    fallback_to_site_default: true
  preferred_admin_langcode:
    plugin: user_langcode
    source: language
    fallback_to_site_default: true
  init: init
  roles:
    plugin: migration_lookup
    migration: d7_user_role
    source: roles
  user_picture:
    -
      plugin: default_value
      source: picture
      default_value: null
```

```
    -
      plugin: migration_lookup
      migration: d7_file
destination:
  plugin: entity:user
migration_dependencies:
  required:
    - d7_user_role
  optional:
    - d7_field_instance
    - d7_file
    - language
    - default_language
    - user_picture_field_instance
    - user_picture_entity_display
    - user_picture_entity_form_display
```

Again, we have ID of the migration, label and migration tags.

```
id: d7_user
label: User accounts
migration_tags:
  - Drupal 7
class: Drupal\user\Plugin\migrate\User
```

User migration has its own migrate plugin class User which extends FieldMigration. This is needed to migrate the custom fields that we might have added to the User entity on the Drupal 7 source site. We don't get deeper to this topic in this tutorial but you are, of course, free to investigate the source code of the plugin class.

Defining the source plugin

```
source:
  plugin: d7_user
```

We're defining here that we are using the User source plugin. The id ('d7_user') we use in the migration matches to the plugin annotation of the plugin class.

This source plugin will query the source data from the D7 database table 'users'. For implementation details, see User::query().

Defining the migration process and transformations

```
process:
  uid: uid
  name: name
  pass: pass
  mail: mail
  created: created
  access: access
  login: login
  status: status
  timezone: timezone
  langcode:
    plugin: user_langcode
    source: language
    fallback_to_site_default: false
  preferred_langcode:
    plugin: user_langcode
    source: language
    fallback_to_site_default: true
  preferred_admin_langcode:
    plugin: user_langcode
    source: language
    fallback_to_site_default: true
  init: init
  roles:
    plugin: migration_lookup
    migration: d7_user_role
    source: roles
  user_picture:
    -
      plugin: default_value
      source: picture
      default_value: null
    -
      plugin: migration_lookup
      migration: d7_file
```

We have here a lot of 1:1 field mapping, for example D8 'uid' is mapped 1:1 from D7 'uid', 'name' is mapped 1:1 from 'name' and so on.

The langcode mapping looks a bit different, let's have a look at that.

```
langcode:
  plugin: user_langcode
  source: language
  fallback_to_site_default: false
```

- We are first saying that we're talking about the transformation of the D8 destination property 'langcode'.
- The process plugin to be used for this is UserLangcode
- This is a process plugin provided by the User module itself.
- The process plugin class must be created in the MODULE/src/Plugin/migrate/process directory
- The plugin ID used in the migration must be found in the plugin annotation of the class
- We're saying that the source is coming from the source property 'language'
- The UserLangcode process plugin takes another argument 'fallback_to_site_default' which is set to 'false'.

The real transformation logic is defined in UserLangcode::transform(). If we look at the source of this transform method, it contains the following logic:

- If the user's language is empty and the 'fallback_to_site_default' is 'false', it means the locale module was not installed and the user's langcode should be set to English.
- Then we check that the user's language actually exists in Drupal 8. If it does not exist in Drupal 8, we use the default language of the Drupal 8 site.

- If the language was found, we use that as user's language.

Let's next have a closer look at the user role mapping:

```
roles:
  plugin: migration_lookup
  migration: d7_user_role
  source: roles
```

- We are first saying that we're talking about the D8 'roles' that the user should be given.
- In order to be able to give roles to the user, the roles must have been migrated already earlier using the 'd7_user_role' migration.

- We use the migration_lookup process plugin which takes the D7 roles as an input and returns the corresponding D8 role ids.

The migration_lookup process plugin is one of the most fundamental process plugins.

Let's then have a closer look at the processing for user picture.

```
user_picture:
  -
    plugin: default_value
    source: picture
    default_value: null
  -
    plugin: migration_lookup
    migration: d7_file
```

Here the Drupal 7 source value 'picture' must pass through two process plugins to end up with the correct Drupal 8 value. The anatomy of the process pipeline is explained on the migrate process overview documentation page.

What happens here is as follows:

- We're first passing the Drupal 7 source property 'picture' through the default_value process plugin. This property is a 1:1 relationship with the Drupal 7 `file_managed.fid` value.
- This plugin passes the given default value (null in this case) to the next phase of the process pipeline if the original source property had no value (NULL, zero or empty string). If the source property has a value, then that value is preserved and passed to the next phase of the process pipeline.
- The second phase of the process pipeline is the migration_lookup plugin which we already covered above in this article. It uses the value from the previous pipeline step as its source and checks the ID of the file which was previously migrated using the d7_file migration.

Defining the destination plugin

```
destination:
  plugin: entity:user
```

Here we are saying that the migration is generating Drupal 8 user entities.

Migration dependencies

```
migration_dependencies:
  required:
    - d7_user_role
  optional:
```

```
  - d7_field_instance
  - d7_file
  - language
  - default_language
  - user_picture_field_instance
  - user_picture_entity_display
  - user_picture_entity_form_display
```

We are declaring here that the migration d7_user_role is a prerequisite for running this migration. The optional migrations do not necessarily have to be executed but Migrate Drupal will execute them before this one if you are running all migrations at one go.

Advanced example: Node migration uses deriver class to dynamically build migrations for each content type

The source Drupal 6 or Drupal 7 site will have the content types the site builder had created and all content types have their own fields.

- There might be for example a content type *Car* with fields *make* and *model*.
- And content type *Driver* with fields *name* and *age*.

What we need to achieve is obviously to migrate all Drupal 7 Car nodes to Drupal 8 as Car nodes and all Drupal 7 Driver nodes to Drupal 8 as Driver nodes. If we inspect the migrations at admin/config/development/configuration/single/export after the migrations have been generated (select 'Migration' as 'Configuration type'), we can see that there is a separate migration generated for Car nodes and Driver nodes and these two migrations have mappings for the correct fields. Let's have a look at the `core/modules/node/migrations/d7_node.yml` to investigate how did this magic happen.

```
id: d7_node
label: Nodes
migration_tags:
  - Drupal 7
deriver: Drupal\node\Plugin\migrate\D7NodeDeriver
source:
  plugin: d7_node
```

```
  process:
    # If you are using this file to build a custom migration consider removing
    # the nid and vid fields to allow incremental migrations.
    # In D7, nodes always have a tnid, but it's zero for untranslated nodes.
    # We normalize it to equal the nid in that case.
    # @see \Drupal\node\Plugin\migrate\source\d7\Node::prepareRow().
    nid: tnid
    vid: vid
    langcode:
      plugin: default_value
      source: language
      default_value: "und"
    title: title
    uid: node_uid
    status: status
    created: created
    changed: changed
    promote: promote
    sticky: sticky
    revision_uid: revision_uid
    revision_log: log
    revision_timestamp: timestamp
  destination:
    plugin: entity:node
  migration_dependencies:
    required:
      - d7_user
      - d7_node_type
    optional:
      - d7_field_instance
      - d7_comment_field_instance
```

The secret lies in the deriver class which is D7NodeDeriver and to be more specific, in D7NodeDeriver::getDerivativeDefinitions(). Since this is a very advanced topic, reading the source is the best way to understand the flow. On high level:

- We first read all node types that the site has. This is needed to generate the different migrations for each content type (i.e. Cars and Drivers in the example above)
- For each node type, we generate the field mappings based on the fields that each node type has.
- And finally we return the derivatives.

Field migrations: using email field migration as an example

Writing an upgrade path for a module providing a field type is not too complex. We'll use the Drupal 7 Email as an example. The Email module was moved to Drupal 8 core but the example is valid also for modules which remain in contrib space for Drupal 8.

Write a plugin class that extends FieldPluginBase

- Class \Drupal\Core\Field\Plugin\migrate\field\Email extends \Drupal\migrate_drupal\Plugin\migrate\field\FieldPluginBase
- FieldPluginBase implements MigrateFieldInterface

- The namespace to use for your plugin class is `Drupal\MODULE\Plugin\migrate\field` and thus the file should be located in `MODULE/src/Plugin/migrate/field directory` (where MODULE is obviously the name of your module).

Let's start by inspecting the annotation of this plugin class

```
@MigrateField(
  id = "email",
  core = {6,7},
  type_map = {
    "email" = "email"
  },
  source_module = "email",
  destination_module = "core"
)
```

- ID and core versions are straight forward.
- The field type names are 'email' in both Drupal 8 and Drupal 6/7 (left hand side of the 'type_map' is the Drupal 8 field type and right hand side is the Drupal 6/7 field type.

- 'source_module' and 'destination_module' define the module names that are shown in the Migrate Drupal UI. Note: These became required as part of #2859304: Show field type migrations correctly in Migrate Drupal UI, see change record.

Methods to implement

The methods the plugin class need to implement depend obviously on the field that is being migrated. Email field is quite typical and it implements getFieldWidgetMap(), getFieldFormatterMap() and defineValueProcessPipeline().
Widget and formatter maps are straight forward array maps:

```
/**
 * {@inheritdoc}
 */
public function getFieldWidgetMap() {
  return [
    'email_textfield' => 'email_default',
  ];
}
```

- The email field migration maps Drupal 6/7 'email_textfield' to Drupal 8 'email_default'.

```
/**
 * {@inheritdoc}
 */
public function getFieldFormatterMap() {
  return [
    'email_formatter_default' => 'basic_string',
    'email_formatter_contact' => 'basic_string',
    'email_formatter_plain' => 'basic_string',
    'email_formatter_spamspan' => 'basic_string',
  ];
}
```

- The email field migration maps all different Drupal 6/7 field formatter settings to Drupal 8 'basic_string'.

```
/**
 * {@inheritdoc}
 */
public function defineValueProcessPipeline(MigrationInterface $migration,
$field_name, $data) {
  $process = [ 'plugin' => 'iterator',
    'source' => $field_name,
    'process' => [
      'value' => 'email',
    ],  ];
  $migration->setProcessOfProperty($field_name, $process);
}
```

- The actual processing of field values is done by implementing processFieldValue()
- The email field migration uses the iterator process plugin which allows migration of multi-value fields.
- Note: iterator process plugin is deprecated. Use SubProcess process plugin instead.
- For each value we apply mapping so that the Drupal 8 destination property 'value' is mapped from Drupal 6/7 source property 'email'.

Summary and further reading

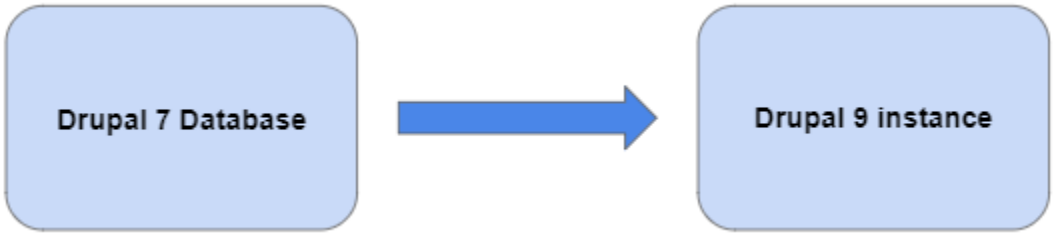
The examples above demonstrate different aspects of how Drupal core modules are migrating to Drupal 8.

- Migrate source plugins page has more information on how to write your own source plugin if you need to define one.
- Migrate process plugins page has more information on the general purpose process plugins provided by Drupal 8 core and how to define your own process plugins.
- Migrate destination plugins page has more information on the destination plugins.

A SIMPLE GUIDE TO A DRUPAL 7 TO DRUPAL 9 MIGRATION FROM A DATABASE SOURCE

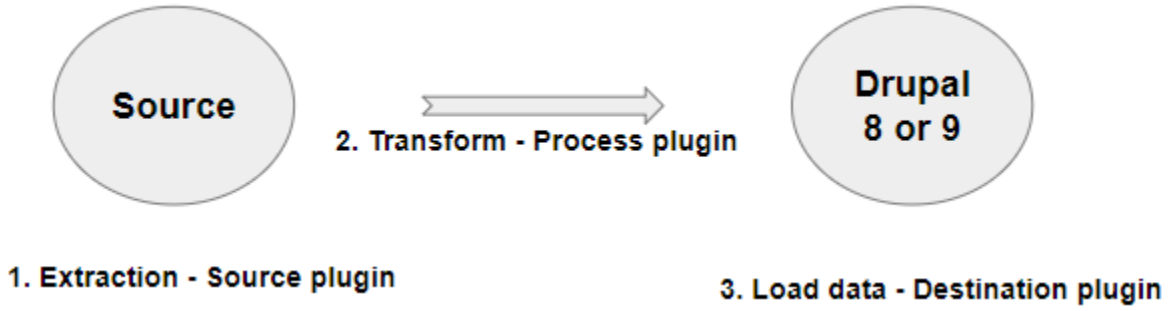
What is a Drupal Migration?

Migration is a movement of content from Drupal 7 to Drupal 9. This includes configuration (content types, vocabularies, user roles, file types, image styles, etc.) and contents (users, nodes, paragraphs, taxonomy terms etc.)



The ETL Process

No matter what your source or destination is for a migration, a Drupal migration will follow the ETL process or the **Extract – Transform – Load** process. The first step is the **Extraction** step where the data is extracted from the source plugin. The next step is the Transformation step where the extracted data is processed according to the requirements by the process plugin. And lastly, the Loading step where the data is loaded into the storage by the destination plugin.



Migration Plugins and Modules

1. Source plugin

A set of data, called row from the source. They help fetch data from the source like a database, CSV, XML or JSON.

2. Process plugin

Row data will be processed and transformed as required. It describes how the destination is to be built from the source data.

3. Destination plugin

Once processed, the transformed row is saved to the destination in the Drupal site.

Modules Required

Migrate Module – Contains APIs to migrate content and configuration to Drupal 9.
Migrate Drupal Module – Helps in migrating content and configuration from a Drupal source site to Drupal 9.
Migrate Drupal UI Module – A user interface to perform the migration.
Migrate plus Module – Provides capabilities to transform source data and APIs for grouping migrations.
Migrate tools Module – A contributed module that offers extended drush commands to manage Drupal 9 migrations.
Drupal upgrade – A contributed module used to create the migration YML scripts.

How do I Migrate to Drupal 9: The Groundwork

Now that we have a fairly good understanding of some of the basic migration concepts and have installed the basic modules needed for the migration (as discussed above), let's look at a few simple steps to follow before actually commencing a Drupal 7 to Drupal 9 migration:

Audit the Drupal 7 site for contributed modules. Many of the fields in the Drupal 7 content types will depend on contributed modules. So, we will need to check for those modules and install the same in Drupal 9.

Install the contributed modules if it is not incorporated in Drupal 9 core.

Install the supportive core modules like revisions, translations, etc. if needed.

Have the Drupal 7 database and files accessible by the Drupal 9 instance.

Select the method of migration Bulk or Individual carefully based on the complexity of the site.

Connecting to the Drupal 7 Database

Now we will need to connect to the source database of the Drupal 7 website. For that, we will need to add the source database information in the Drupal 9 settings file.

Go to **settings.php** file of your Drupal 9 site and add the database with migrate key and run this command:

```
drush migrate-upgrade --configure-only --legacy-db-key=migrate --
legacy-root=https://www.drupal7.com/
```

Or you could also directly configure using this URL:

```
drush migrate-upgrade --configure-only
--legacy-db-url=mysqli://drupaluser@127.0.0.1:33067/drupal7_db --
legacy-root=https://drupal7.com/
```

The above commands will generate migration YMLs based on the Drupal 7 configuration and contents. It will not start the migration as we added the option **“--configure-only”**.

Drupal 7 to Drupal 9 Migration - Bulk import

To perform a bulk migration on all configurations and contents to Drupal 9, you will need to execute these commands :

`drush migrate:import --tag=Configuration --execute-dependencies`
This will run all the configuration migrations by running the dependencies at first.

`drush migrate:import --tag=Content --execute-dependencies`

This command will run all the content migrations by running the dependencies at first. Please Note: The migration order is important.

Please Note: The migration order is important.

Drupal 7 to Drupal 9 Migration - Individual import

Sometimes, you might not want to migrate ALL of your content of configurations at one shot. You can then opt to migrate them individually. However, you will need to keep in mind to migrate them in order.

- Configuration: User roles, vocabularies, content types, image styles, fields, field instances etc.
- Content: Files, paragraphs, Users, terms, nodes, etc.

Migration Tools drush Commands

The Migration Tools module (as discussed previously) is a contributed module that gives you some tools and commands that will help you during a migration. Some of the commands include:

drush ms - Migration status

drush mim <migration> - Migration import

Imports the data from Drupal. We can also run the import with a limit. For example:

drush mim --limit=10

drush mr <migration> - Migration rollback

drush mmsg <migration> - Migration message

Shows the captured message (error or notice) after the migration import.

How to implement Queue Worker/API in Drupal 8?

Before I start my writing about Queue Worker, it is important to understand about cron job in Drupal 8.

In generic terms, cron is a scheduled job which runs periodically at fixed intervals. cron manages typical tasks such as database maintenance, sending bulk emails, fetching data from a third-party on regular basis.

It manages short running tasks with fewer resources and it will automatically time-out while performing time-consuming tasks such as fetching data from a third- party.

In Drupal you can write cron like:-

```
function hook_cron() {
  //code.
}
```

in .module file of your module.

So let’s take an example:-
There is a module namely – my_module.
so in **my_module.module** file write

```
function my_module_cron() {
	$mailManager = \Drupal::service(‘plugin.manager.mail’);
	$mailManager->mail($module, $key, $to, $langcode, $params = array(), $reply = NULL, $send =
TRUE);
}

/**
* Implements hook_mail().
*/
function my_module_mail($key, &$message, $params) {
	switch ($key) {
	   case ‘key’:
		$message[‘subject’] = t(‘Test’);
		$message[‘body’][] = ‘Hello World!’;
		break;
	   }
}
```

Whenever cron is executed it will send an email.

drush commands to run a cron manually.

drush core-cron

alias

drush cron

Disadvantages:-

1. You can not run an individual cron.
2. All hook_cron runs at same time for all modules according to module weight

Queue Worker A Powerful manual cron system.

3. When cron fails for a module, the other cron of different modules will be not executed.
4. There is no logging information that can tell us which module was the culprit for cron failure.
5. Cron can’t handle large and complex tasks in huge quantity.
6. You can’t run cron with time limits.

QueueWorker:-

The following disadvantages compel us to use queue API of Drupal 8 which is known as Queue Worker.

Queue Worker offers a lot of advantages such as :

1. Offers flexibility to run queue with time limits
2. You can individual queue at one time as compared to cron which runs all the cron
3. Efficient as compared to cron, can handle resource intensive tasks
4. If any failure occurs, you can revert the item back to queue to process that again
5. Multiple queues can run without interrupting other work
6. You can log for each item of queue easily

Queue Worker rules out the disadvantage of hook_cron and hence solve the issues with hook cron very efficiently.

Here are some of the basics concepts of Queue API along with an example:

The Queue API in Drupal allows you to complete tasks at a later stage. We can insert items in the queue and process them when we require. We can run Queue either by running cron or by manually using Drush commands.

Some very basic concepts of Queue working is FIFO.

Components of Queue API.

1. **QueueInterface**:-
   - It plays a very important role here. Its implementation represents queue, which means the object of a class that implements QueueInterface.
2. **DatabaseQueue**:-
   - It is a reliable type of queue which makes ensure that it does not timeout. All the items are processed without any error and in (FIFO) order.
3. **Roles of Queue Object**:-
   - The typical role of the queue object is to create items, claim them from the queue and delete them when they have been processed.
   - You can also release the items if processing is not finished or another Queue needs to be processed before removing them from Queue.

Here’s an example of Queue Worker.

You have a user query form on your website. See the attached screenshot of Form. So, whenever a user submits a query instead of sending mail at run-time we will store these details in Queue Worker and later run Queue Worker manually whenever we want to receive emails.

Folder Structure. See attached screenshot.

```
sarthak@sarthak /var/www/html/drupal/modules $ tree my_module/
my_module/
|-- my_module.info.yml
|-- my_module.module
|-- my_module.routing.yml
`-- src
    |-- Form
    |   `-- UserQueryForm.php
    `-- Plugin
        `-- QueueWorker
            |-- CronEventProcessor.php
            `-- EmailEventBase.php

4 directories, 6 files
sarthak@sarthak /var/www/html/drupal/modules $
```

my_module.info.yml

name: My Module

description: ‘Example how to use Queue Worker’

package: Custom

type: module

core: 8.x

my_module.routing.yml

my_module.query_form:

path: ‘/user-query-form’

defaults:

_form: ‘\Drupal\my_module\Form\UserQueryForm’

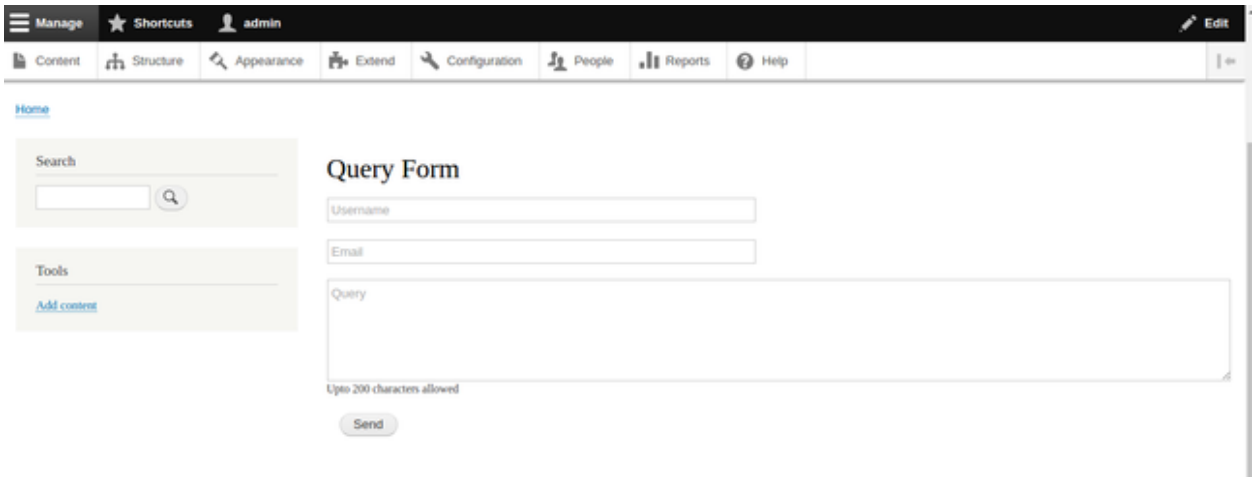
_title: ‘Query Form’

requirements:

_permission: ‘access content’

UserQueryForm.php

Screenshot:



Code:-

```
namespace Drupal\my_module\Form;

use Drupal\Core\Form\FormBase;

use Drupal\Core\Form\FormStateInterface;
```

```
/**
* Class UserQueryForm.
*
* @package Drupal\my_module\Form
*/
class UserQueryForm extends FormBase {
/**
* {@inheritdoc}
*/
public function getFormId() {
 return 'user_query_form';
}
/**
* {@inheritdoc}
*/
public function buildForm(array $form, FormStateInterface $form_state) {
$form['name'] = [
 '#type' => 'textfield',
 '#attributes' => [
   'placeholder' => 'Username',
   ],
 '#required' => true,  ];
```

```
$form['email'] = [
 '#type' => 'email',
 '#attributes' => [
 'placeholder' => 'Email',
 ],
 '#required' => true,  ];
$form['query'] = [
 '#type' => 'textarea',
 '#attributes' => [
 'placeholder' => 'Query',
 ],
 '#description' => 'Upto 200 characters allowed',
 '#required' => true,  ];
$form['submit'] = [
 '#type' => 'submit',
 '#value' => 'Send',];
return $form;
}
/**
* {@inheritdoc}
*/
public function validateForm(array &$form, FormStateInterface $form_state) {
```

```
}

/**

* {@inheritdoc}

*/

 public function submitForm(array &$form, FormStateInterface $form_state) {

 /** @var QueueFactory $queue_factory */

 $queue_factory = \Drupal::service('queue');

 /** @var QueueInterface $queue */

 $queue = $queue_factory->get('email_processor');

 $item = new \stdClass();

 $item->username = $form_state->getValue('name');

 $item->email = $form_state->getValue('email');

 $item->query = $form_state->getValue('query');

 $queue->createItem($item);

 }

}
```

First create a `QueueFactory` object from the service container statically and use this object to instantiate a queue object of email_proceesor.

Here we are calling createItem() function to insert items in Queue.

So, whenever a user submits a query form an item will be inserted in the Queue.

To see the items in Queue use this drush command:

drush queue-list

EmailEventBase.php

The Queue Worker

The Queue Worker is a Plugin so we create **EmailEventBase.php** inside Plugin Folder as attached screenshot of folder structure.

```
/**
* Handles sending of email.
*
* PHP Version 5
*/

namespace Drupal\my_module\Plugin\QueueWorker;

use Drupal\Core\Plugin\ContainerFactoryPluginInterface;
use Drupal\Core\Queue\QueueWorkerBase;
use Drupal\Core\Mail\MailManager;
use Symfony\Component\DependencyInjection\ContainerInterface;

/**
*
* @inheritdoc
*/
class EmailEventBase extends QueueWorkerBase implements ContainerFactoryPluginInterface {

/**
*
* @var Drupal\Core\Mail\MailManager
```

```
*/
protected $mail;
/**
* constructor
*/
public function __construct(MailManager $mail) {
	$this->mail = $mail;
}

/**
* {@inheritdoc}
*/
public static function create(ContainerInterface $container, array $configuration, $plugin_id,
$plugin_definition) {
	return new static(
		$container->get('plugin.manager.mail')
	);
}

/**
* Processes a single item of Queue.
*
*/
  public function processItem($data) {
	$params['subject'] = t('query');
	$params['message'] = $data->query;
	$params['from'] = $data->email;
	$params['username'] = $data->username;
	$to = \Drupal::config('system.site')->get('mail');
	$this->mail->mail('my_module','query_mail',$to,'en',$params,NULL,true);
  }
}
```

In processItem() function we are processing individual items of Queue.

Now, let's create a CronEventProcessor plugin that will use this logic on Cron runs or when we will run manually using drush command:

CronEventProcessor.php

```
namespace Drupal\my_module\Plugin\QueueWorker;

/**
*
* @QueueWorker(
* id = "email_processor",
* title = "My custom Queue Worker",
* cron = {"time" = 10}
* )
*/
class CronEventProcessor extends EmailEventBase {

}
```

In this file, we have written an annotation. So by writing an annotation, we are telling Drupal that whenever cron runs process this queue for 10 seconds.

Whenever cron runs Queue Worker will load all the plugins that cron as their key in annotations. The id in the annotation is the user for creating the object of the queue.

in **.module** file of your module you need to write hook_mail to send mail.

```
/**
* Implements hook_mail()
* @param type $key
* @param type $message
```

```
* @param type $params
*/
function my_module_mail($key, &$message, $params) {
switch ($key) {
case ‘query_mail’:
$body = "Hi ".$params[‘username’]
. " Thanks for posting your Query"
. $params[‘message’]
. "Your username: " . $params[‘username’]

. "Your email: " . $params[’email’];

$message[‘from’] = $params[‘from’];
$message[‘subject’] = $params[‘subject’];
$message[‘body’][] = Drupal\Core\Mail\MailFormatHelper::htmlToText($body);
break;
}
}
```

To run this queue manually we can use drush command like:-

```
drush queue-run email_processor –time-limit=10
```

Conclusion:-

You should use Queue API instead of native hook_cron(). It has many benefits as compared to hook_cron.

Manage your resource intensive tasks on your website using Queue API.